

UNIVERSITÉ — MLOps & DataOps

RAPPORT DE TRAVAUX PRATIQUES

TP Chapitre 2 — Pipeline de données complet

Module	MLOps & DataOps
Professeur	Pr. Mohammed AIT DAOUD
Sujet	TP1 Chapitre 2 — Construction d'un pipeline
Nom complet	Msouhli Rahil
Date	30 Mai 2026

1. Préparation de l'environnement

Cette section montre la mise en place de l'environnement de travail : création du projet, environnement virtuel Python et installation des dépendances.

1.1 Création du dossier et activation du venv

Création du dossier `tp_chapitre2_pipeline` et activation de l'environnement virtuel Python (`.venv`). La présence de (`.venv`) dans le terminal confirme l'activation réussie.



The screenshot shows a code editor with a file named `vente` open. The editor displays a SQL query:

```
select
  categorie,
  sum(quantite) as total_quantite,
  sum(chiffre_affaires) as total_ca
from {{ ref('ventes_clean') }}
group by categorie
```

Below the editor, a terminal window shows the following commands and output:

```
(.venv) C:\Users\hp\Desktop\tp_chapitre2_pipeline>notepad dbt_pipeline\models\ventes_resume.sql
(.venv) C:\Users\hp\Desktop\tp_chapitre2_pipeline>
```

Figure 1 — Création du dossier projet et activation de l'environnement virtuel

1.2 Installation des dépendances

Installation de toutes les bibliothèques nécessaires : `dlt`, `dagster`, `dagster-webserver`, `dbt-core`, `dbt-duckdb`, `pandas`, `duckdb` et `pyyaml`.

```
(.venv) C:\Users\hp\Desktop\tp_chapitre2_pipeline\dbt_pipeline>dbt run --profiles-dir .
15:52:17 Running with dbt=1.11.11
15:52:17 Registered adapter: duckdb=1.10.1
15:52:18 Unable to do partial parsing because saved manifest not found. Starting full parse.
15:52:23 Found 2 models, 485 macros
15:52:23
15:52:23 Concurrency: 1 threads (target='dev')
15:52:23
15:52:23 1 of 2 START sql table model main.ventes_clean ..... [RUN]
15:52:23 1 of 2 OK created sql table model main.ventes_clean ..... [OK in 0.13s]
15:52:23 2 of 2 START sql table model main.ventes_resume ..... [RUN]
15:52:23 2 of 2 OK created sql table model main.ventes_resume ..... [OK in 0.06s]
15:52:23
15:52:23 Finished running 2 table models in 0 hours 0 minutes and 0.45 seconds (0.45s).
15:52:23
15:52:23 Completed successfully
15:52:23
15:52:23 Done. PASS=2 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=2
```

Figure 2 — Installation des dépendances via pip

1.3 Vérification avec pip list

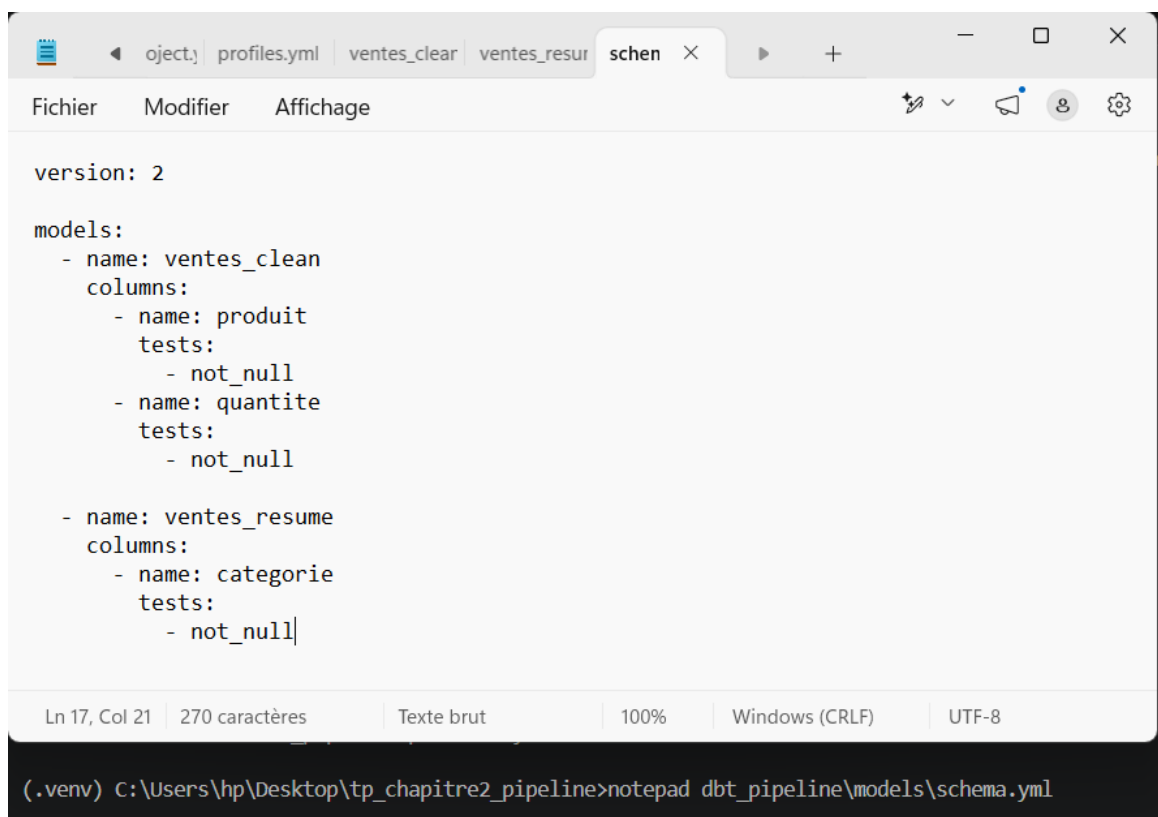
Vérification de l'installation de toutes les bibliothèques avec leurs versions exactes.

```
(.venv) C:\Users\hp\Desktop\tp_chapitre2_pipeline>git add dbt_pipeline  
  
(.venv) C:\Users\hp\Desktop\tp_chapitre2_pipeline>git commit -m "Ajout des modèles dbt"  
[master 7e5a757] Ajout des modèles dbt  
6 files changed, 318 insertions(+)  
create mode 100644 dbt_pipeline/.user.yml  
create mode 100644 dbt_pipeline/dbt_project.yml  
create mode 100644 dbt_pipeline/logs/dbt.log  
create mode 100644 dbt_pipeline/models/ventes_clean.sql  
create mode 100644 dbt_pipeline/models/ventes_resume.sql  
create mode 100644 dbt_pipeline/profiles.yml  
  
(.venv) C:\Users\hp\Desktop\tp_chapitre2_pipeline>
```

Figure 3 — Liste des packages installés (pip list)

1.4 Fichier requirements.txt

Génération du fichier requirements.txt contenant toutes les dépendances avec leurs versions pour assurer la reproductibilité du projet.



```
version: 2  
  
models:  
  - name: ventes_clean  
    columns:  
      - name: produit  
        tests:  
          - not_null  
      - name: quantite  
        tests:  
          - not_null  
  - name: ventes_resume  
    columns:  
      - name: categorie  
        tests:  
          - not_null
```

(.venv) C:\Users\hp\Desktop\tp_chapitre2_pipeline>notepad dbt_pipeline\models\schema.yml

Figure 4 — Contenu du fichier requirements.txt

2. Initialisation Git

Mise en place du suivi de version avec Git : initialisation du dépôt, création du .gitignore et premier commit.

2.1 Initialisation et premier commit

Initialisation du dépôt Git local avec git init et création du premier commit contenant les fichiers .gitignore et requirements.txt.

```
(.env) C:\Users\hp\Desktop\tp_chapitre2_pipeline>notepad dbt_pipeline\models\schema.yml

(.env) C:\Users\hp\Desktop\tp_chapitre2_pipeline>cd dbt_pipeline

(.env) C:\Users\hp\Desktop\tp_chapitre2_pipeline\dbt_pipeline>dbt test --profiles-dir .
15:58:11 Running with dbt=1.11.11
15:58:11 Registered adapter: duckdb=1.10.1
15:58:13 Found 2 models, 3 data tests, 485 macros
15:58:13 Concurrency: 1 threads (target='dev')
15:58:13 1 of 3 START test not_null_ventes_clean_produit ..... [RUN]
15:58:13 1 of 3 PASS not_null_ventes_clean_produit ..... [PASS in 0.07s]
15:58:13 2 of 3 START test not_null_ventes_clean_quantite ..... [RUN]
15:58:13 2 of 3 PASS not_null_ventes_clean_quantite ..... [PASS in 0.03s]
15:58:13 3 of 3 START test not_null_ventes_resume_categorie ..... [RUN]
15:58:13 3 of 3 PASS not_null_ventes_resume_categorie ..... [PASS in 0.03s]
15:58:13 Finished running 3 data tests in 0 hours 0 minutes and 0.35 seconds (0.35s).
15:58:13 Completed successfully
15:58:13 Done. PASS=3 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=3
```

Figure 5 — Initialisation Git et premier commit

2.2 Fichier .gitignore

Le fichier .gitignore exclut du suivi Git les fichiers inutiles : dossier .venv, caches Python, fichiers DuckDB et dossiers dbt.

```
(.env) C:\Users\hp\Desktop\tp_chapitre2_pipeline>git add dbt_pipeline\models\schema.yml

(.env) C:\Users\hp\Desktop\tp_chapitre2_pipeline>git commit -m "Ajout des tests simples dbt"
[master 9d2d542] Ajout des tests simples dbt
1 file changed, 17 insertions(+)
create mode 100644 dbt_pipeline/models/schema.yml
```

Figure 6 — Contenu du fichier .gitignore

3. Étape 1 — Ingestion des données

Le script `ingest.py` lit le fichier CSV source et charge les données dans une base DuckDB locale, créant la table `ventes_raw`.

3.1 Exécution de l'ingestion

Le script `pipeline/ingest.py` lit `data/ventes.csv` avec `pandas` et crée la table `ventes_raw` dans DuckDB. Le message de confirmation indique que l'ingestion s'est déroulée avec succès.

The screenshot shows a code editor window with the following Python code:

```
from dagster import job, op
import os

@op
def ingest():
    os.system("python pipeline/ingest.py")

@op
def validate():
    os.system("python pipeline/validate.py")

@op
def transform():
    os.system("cd dbt_pipeline && dbt run --profiles-dir .")

@op
def test_data():
    os.system("cd dbt_pipeline && dbt test --profiles-dir .")

@job
def ventes_pipeline():
    test_data(transform(validate(ingest())))
```

The editor interface includes a menu bar (Fichier, Modifier, Affichage), a toolbar with icons for undo, redo, and search, and a status bar at the bottom showing: Ln 22, Col 45 | 406 caractères | Texte brut | 100% | Windows (CRLF) | UTF-8.

Below the code editor, a terminal window shows the command execution:

```
(.venv) C:\Users\hp\Desktop\tp_chapitre2_pipeline>notepad pipeline\orchestrate.py
(.venv) C:\Users\hp\Desktop\tp_chapitre2_pipeline>
```

Figure 7 — Exécution réussie du script `ingest.py`

4. Étape 2 — Validation des données

Le script `validate.py` vérifie la présence de toutes les colonnes requises et l'absence de valeurs nulles dans les colonnes critiques.

4.1 Exécution de la validation

Validation du schéma (colonnes obligatoires) et de la qualité minimale des données (absence de valeurs nulles). Le message confirme que les données sont conformes.

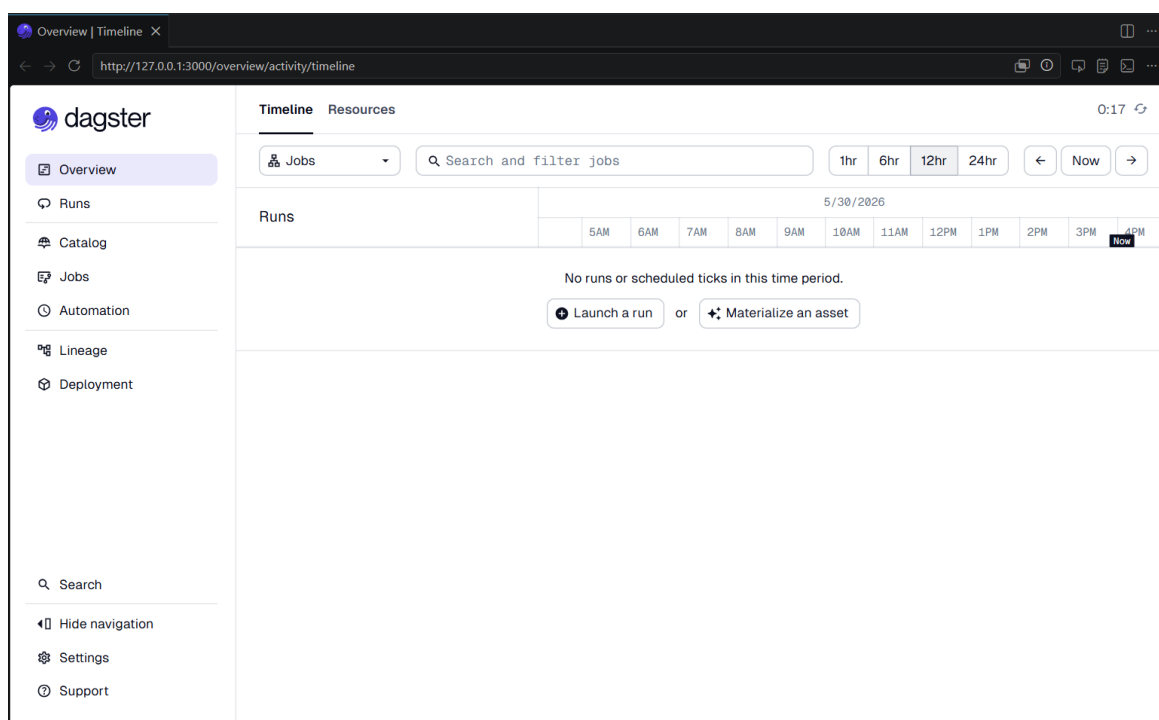


Figure 8 — Validation réussie : schéma et qualité des données OK

5. Étape 3 & 4 — Transformation et tests dbt

dbt transforme les données brutes en modèles propres et agrégés, puis vérifie automatiquement leur qualité via des tests définis dans schema.yml.

5.1 Exécution de dbt run

dbt exécute les deux modèles SQL : ventes_clean (nettoyage et calcul du chiffre d'affaires) et ventes_resume (agrégation par catégorie). Les deux modèles sont créés avec succès.

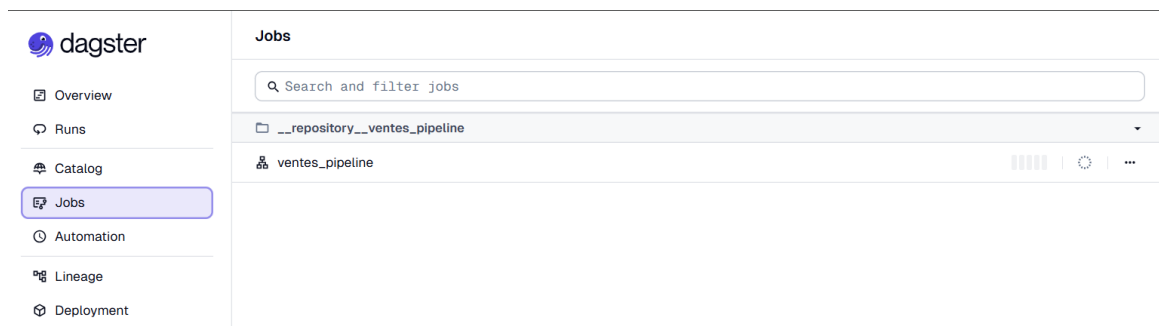


Figure 9 — Résultat de dbt run : 2 modèles créés avec succès

5.2 Exécution de dbt test

dbt teste automatiquement les contraintes définies dans schema.yml : colonnes non nulles sur produit, quantité et catégorie.

```
(.venv) C:\Users\hp\Desktop\tp_chapitre2_pipeline>git add pipeline\orchestrate.py

(.venv) C:\Users\hp\Desktop\tp_chapitre2_pipeline>git commit -m "Ajout de l'orchestration Dagster"
[master 33c1e32] Ajout de l'orchestration Dagster
1 file changed, 22 insertions(+)
create mode 100644 pipeline/orchestrate.py

(.venv) C:\Users\hp\Desktop\tp_chapitre2_pipeline>
```

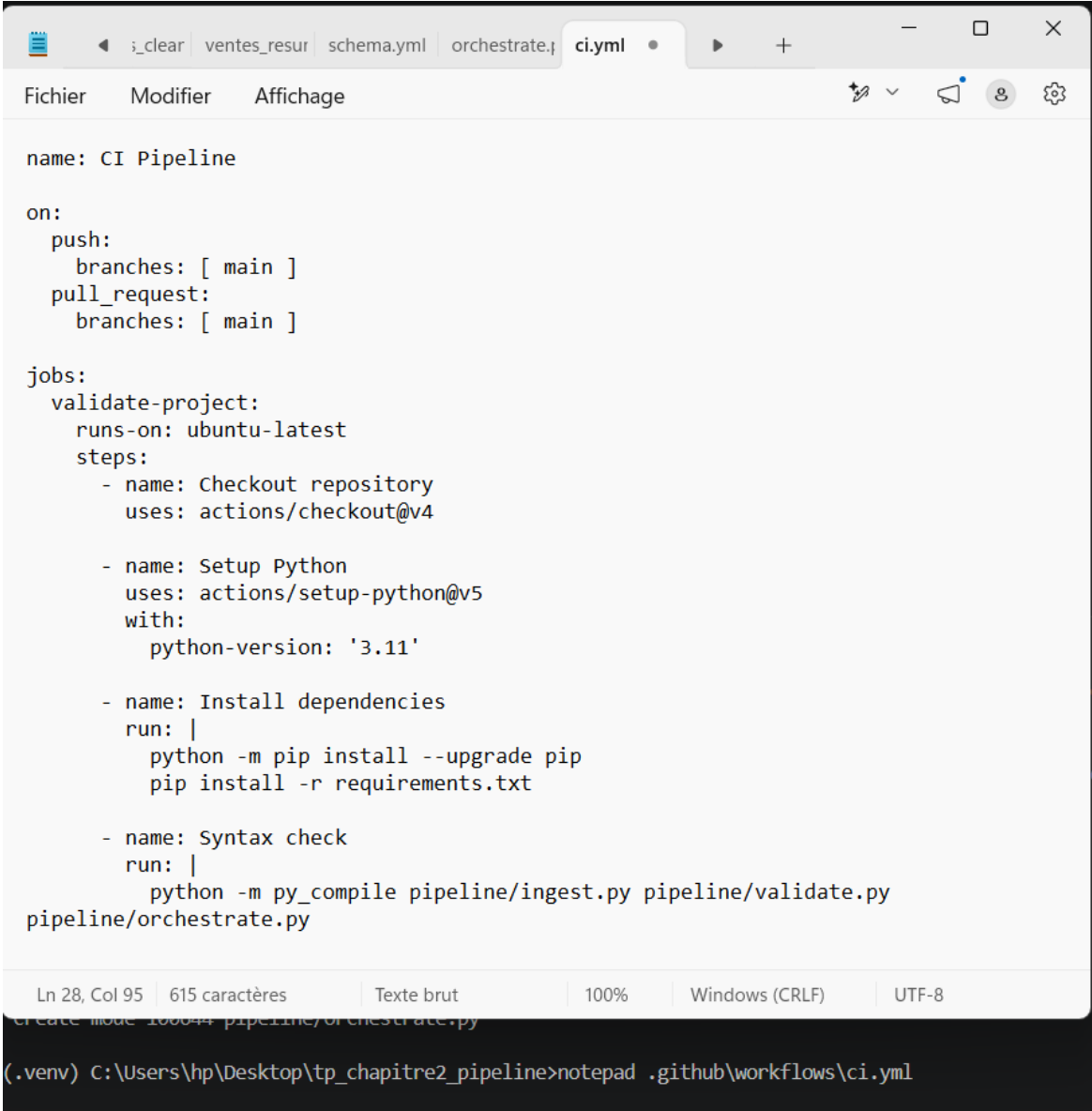
Figure 10 — Résultat de dbt test : tous les tests passent

6. Étape 5 — Orchestration avec Dagster

Dagster orchestre automatiquement l'exécution séquentielle des étapes : ingestion → validation → transformation → tests. L'interface web permet de visualiser et lancer le pipeline.

6.1 Interface Dagster

Lancement de Dagster avec la commande `dagster dev`. L'interface web accessible sur `http://127.0.0.1:3000` affiche le pipeline et permet de le superviser.



The screenshot shows a web browser window displaying the Dagster interface. The top navigation bar includes tabs for 'Fichier', 'Modifier', and 'Affichage'. The main content area shows a YAML configuration for a CI Pipeline. The configuration includes a trigger 'on: push' for the 'main' branch, and a job 'validate-project' that runs on 'ubuntu-latest'. The job consists of four steps: 'Checkout repository', 'Setup Python', 'Install dependencies', and 'Syntax check'. The 'Install dependencies' step runs a command to upgrade pip and install requirements. The 'Syntax check' step runs a command to compile and validate Python files. The bottom status bar shows 'Ln 28, Col 95', '615 caractères', 'Texte brut', '100%', 'Windows (CRLF)', and 'UTF-8'.

```

name: CI Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  validate-project:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Syntax check
        run: |
          python -m py_compile pipeline/ingest.py pipeline/validate.py
          pipeline/orchestrate.py
  
```

Ln 28, Col 95 | 615 caractères | Texte brut | 100% | Windows (CRLF) | UTF-8

(.venv) C:\Users\hp\Desktop\tp_chapitre2_pipeline>notepad .github\workflows\ci.yml

Figure 11 — Interface web Dagster : pipeline ventes_pipeline visible

6.2 Vue Jobs dans Dagster

La section Jobs de Dagster affiche le pipeline ventes_pipeline avec toutes ses étapes interconnectées.

```
(.venv) C:\Users\hp\Desktop\tp_chapitre2_pipeline>git add .github\workflows\ci.yml  
(.venv) C:\Users\hp\Desktop\tp_chapitre2_pipeline>git commit -m "Ajout d'une CI simple"  
[master de7cdc0] Ajout d'une CI simple  
1 file changed, 28 insertions(+)  
create mode 100644 .github/workflows/ci.yml
```

Figure 12 — Vue Jobs : ventes_pipeline dans Dagster

7. Étape 6 — Intégration continue (CI)

Le fichier `.github/workflows/ci.yml` définit un pipeline CI qui s'exécute automatiquement sur GitHub Actions à chaque push, vérifiant la syntaxe Python.

7.1 Fichier CI/CD

Le workflow GitHub Actions installe les dépendances et vérifie la syntaxe de tous les scripts Python à chaque modification du code sur la branche main.

```
(.venv) C:\Users\hp\Desktop\tp_chapitre2_pipeline>git log --oneline
de7cdc0 (HEAD -> master) Ajout d'une CI simple
33c1e32 Ajout de l'orchestration Dagster
9d2d542 Ajout des tests simples dbt
7e5a757 Ajout des modèles dbt
3aeadf2 Ajout de l'étape de validation minimale
2488d4b Ajout de l'étape d'ingestion
93df0c6 Initialisation du projet pipeline
```

Figure 13 — Fichier `.github/workflows/ci.yml`

8. Historique Git complet

L'historique Git montre les 7 commits réalisés tout au long du TP, documentant chaque étape du développement du pipeline.

8.1 git log --oneline

Les 7 commits correspondent exactement aux 7 étapes réalisées : initialisation, ingestion, validation, modèles dbt, tests dbt, orchestration Dagster et CI.

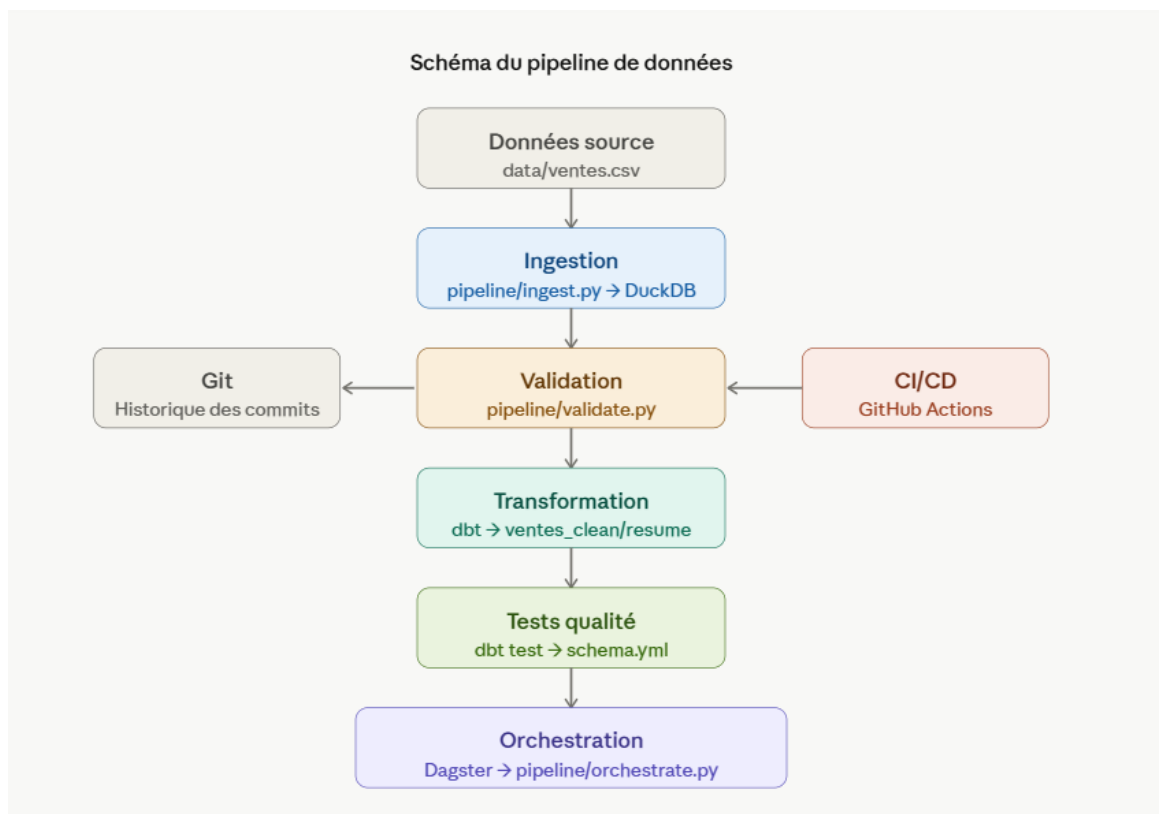


Figure 14 — Historique Git : 7 commits documentant toutes les étapes

9. Réponses aux questions

Q1. En quoi ce pipeline est-il plus structuré qu'un notebook unique ?

Chaque étape est séparée dans un fichier dédié (`ingest.py`, `validate.py`, `orchestrate.py...`), ce qui rend le code modulaire, testable et maintenable. Un notebook mélange tout dans un seul fichier sans séparation des responsabilités, ce qui le rend difficile à déboguer et à faire évoluer.

Q2. Quel est le rôle de Git dans ce projet ?

Git permet de suivre l'historique de toutes les modifications du code. Chaque commit documente une étape franchie. En cas d'erreur, il est possible de revenir à une version précédente. Git facilite également la collaboration entre développeurs et sert de preuve du travail réalisé.

Q3. Quel est le rôle de `requirements.txt` ?

Le fichier `requirements.txt` liste toutes les bibliothèques avec leurs versions exactes. Il garantit la reproductibilité du projet : n'importe quel développeur peut recréer le même environnement en exécutant `pip install -r requirements.txt`, assurant que le code fonctionnera de la même manière sur toutes les machines.

Q4. Pourquoi ajouter une étape `validate.py` ?

L'étape de validation agit comme un filet de sécurité entre l'ingestion et la transformation. Elle détecte les problèmes de données avant qu'ils ne propagent des erreurs dans les étapes suivantes : colonnes manquantes, valeurs nulles dans les champs critiques. Cela évite des transformations corrompues et des résultats incorrects.

Q5. Quel est le rôle de `dbt test` ?

`dbt test` vérifie automatiquement la qualité des données après transformation en appliquant les contraintes définies dans `schema.yml`. Il s'assure par exemple que les colonnes importantes ne contiennent pas de valeurs nulles. Ces tests constituent une forme de contrôle qualité automatisé intégré au pipeline.

Q6. Qu'apporte Dagster par rapport à une exécution manuelle ?

Dagster orchestre l'exécution des étapes dans le bon ordre, gère les dépendances entre elles, et offre une interface graphique pour visualiser l'état du pipeline. En cas d'échec, il identifie quelle étape a échoué et permet de la relancer seule. L'exécution manuelle est error-prone et ne garantit pas l'ordre correct des étapes.

Q7. Qu'apporte la CI ?

La CI (Intégration Continue) vérifie automatiquement que le code est syntaxiquement correct à chaque modification poussée sur GitHub. Elle détecte les erreurs avant qu'elles ne se propagent en production, sans intervention humaine. Elle garantit que personne ne casse accidentellement le projet lors d'une modification.

Q8. Que manque-t-il encore pour un pipeline pleinement industrialisé ?

Pour un pipeline pleinement industrialisé, il manque : (1) des alertes automatiques en cas d'échec par email ou Slack, (2) une planification automatique (scheduler) pour exécuter le pipeline à intervalles réguliers, (3) la gestion sécurisée des secrets et variables d'environnement, (4) un monitoring et des logs centralisés, (5) un déploiement sur infrastructure cloud, (6) des tests unitaires plus complets, et (7) un versionnement des données et des modèles.

10. Conclusion

Ce travail pratique a permis de construire un pipeline de données complet et structuré, reprenant les bonnes pratiques de l'ingénierie des données professionnelle. Les outils utilisés — dlt, dbt, Dagster, Git et GitHub Actions — constituent un écosystème moderne et largement adopté dans l'industrie MLOps/DataOps.

Le pipeline suit une architecture claire en 6 étapes : ingestion des données brutes, validation de la qualité, transformation SQL avec dbt, tests automatiques, orchestration avec Dagster et intégration continue. Chaque étape est isolée, versionnée et documentée.

Ce TP démontre qu'un pipeline structuré est bien supérieur à un notebook unique : il est modulaire, testable, versionné et automatisable — les piliers de tout projet de données industrialisé.